

Unitree H1-2 Robot Telemetry, XR Control, and Digital Twin Platform

Initial executive presentation draft
Prepared for leadership review

Initial executive presentation draft

1. Executive Summary

This repository is a working operator platform for the Unitree H1-2 humanoid robot.

It combines a local web dashboard, live robot telemetry, guarded command endpoints, Vision Pro / XR teleoperation services, camera streaming, full-body telemetry recording, replay visualization, CSV export, deployment automation, and offline validation tooling.

The current system is already useful for supervised robot operation, operator demos, debugging, data capture, and early imitation-learning dataset preparation.

2. What The Platform Delivers

- Live browser dashboard for robot state and supervision.
- Unitree DDS telemetry ingestion from rt/lowstate.
- RH56BFX / Inspire hand telemetry from rt/inspire/state.
- 3D H1-2 digital twin rendered from URDF and STL assets.
- Teleimager camera access and Vision Pro / XR teleoperation entry points.
- Guarded H1 locomotion controls through Unitree LocoClient.
- Guarded right-wrist control experimentation.
- Full-body telemetry recorder for legs, waist, arms, and hands.
- Replay viewer with live reference robot and recorded robot overlay.
- Pose point capture and interpolation replay.
- CSV view and CSV export tooling for analysis workflows.
- Robot-side systemd deployment for repeatable startup.

The repository is more than a frontend project.

It consolidates:

- Dashboard server and static frontend.
- H1-2 robot model assets.
- Vision Pro / XR teleoperation integration.
- Robot deployment service files and patch scripts.
- Semantic teleoperation simulation and execution workspaces.
- Unitree SDK2 Python vendor snapshot.
- RH56BFX hand utilities.
- Robot motion diagnostics.
- Production gate and contract tests.

This gives the team one operational workspace for dashboard development, robot-side deployment, XR experiments, data capture, and simulation support.

Operator browser:

- Opens the dashboard over HTTP.
- Receives live state via /api/state and /events.
- Loads the H1-2 model, Three.js viewer, and dashboard assets.
- Sends explicit operator commands to guarded POST endpoints.

Robot-side server:

- Subscribes to Unitree DDS topics.
- Normalizes robot telemetry.
- Serves dashboard, API, camera bridge, recordings, and model assets.
- Owns command validation and safety gates.

Robot services:

- robot-telemetry-web.service
- teleimager.service
- inspire-hands.service
- xr-teleop.service
- xr-home-watchdog.service

The current dashboard includes these operator pages:

- Dashboard
- Camera Stream
- Teleoperation
- ROS Graph
- Motor Data
- Recorder
- Loco Control
- Right Wrist
- CSV View

The UI is intentionally plain HTML, CSS, and browser JavaScript. There is no frontend build step and no npm install requirement, which makes robot-side deployment simpler.

The platform reads and displays:

- Connection state.
- Snapshot age.
- Sample rate.
- Total sample count.
- Motor count.
- Latest telemetry error.
- Robot metadata: version, mode_pr, mode_machine, tick, crc, and wireless remote state.
- IMU data: quaternion, gyroscope, accelerometer, roll / pitch / yaw, and temperature.
- Battery / BMS values when exposed by firmware.
- Foot force and estimated foot force arrays.
- RH56BFX / Inspire hand connection, sample count, joint count, and finger positions.

The system has a named H1-2 motor contract for 27 body joints:

- Left leg: hip yaw, hip pitch, hip roll, knee, ankle pitch, ankle roll.
- Right leg: hip yaw, hip pitch, hip roll, knee, ankle pitch, ankle roll.
- Waist: waist yaw.
- Left arm: shoulder pitch, shoulder roll, shoulder yaw, elbow, wrist roll, wrist pitch, wrist yaw.
- Right arm: shoulder pitch, shoulder roll, shoulder yaw, elbow, wrist roll, wrist pitch, wrist yaw.

Reserved motor slots are handled explicitly as `ReservedMotorSlot<N>`.

The RH56BFX / Inspire hand contract tracks 12 named hand joints:

- RightPinky
- RightRing
- RightMiddle
- RightIndex
- RightThumbBend
- RightThumbRotation
- LeftPinky
- LeftRing
- LeftMiddle
- LeftIndex
- LeftThumbBend
- LeftThumbRotation

The dashboard treats body telemetry and hand telemetry as independent streams, so the body can remain connected even when the hand bridge is unavailable.

The live digital twin uses:

- static/models/h1_2_description/h1_2.urdf
- Unitree H1-2 STL mesh assets.
- Vendored Three.js modules.

Viewer capabilities:

- H1-2 mesh rendering.
- Body motor telemetry mapped to URDF joints.
- RH56BFX hand telemetry mapped to finger joints.
- Orbit and zoom controls.
- View cube presets: front, back, left, right, top, bottom.
- Grid toggle.
- Auto-rotate toggle.
- Mesh loaded / failed status handling.

Camera features:

- Embedded TeleImager WebRTC preview.
- Direct camera page link.
- Certificate trust guidance for browser access.
- Camera backend state through /api/camera.
- Optional MJPEG bridge at /camera.mjpg.

XR features:

- Vision Pro / Vuer page link.
- Robot-hosted XR service on port 8012.
- Teleoperation method selection:
- Floating VR Control Pad.
- Head Rotation Control.
- Position Matching.
- XR mode changes through /api/xr/mode.

The dashboard exposes guarded Unitree H1 LocoClient controls:

- Ready / Balance Stand.
- Stand Up.
- Start.
- Stop Move.
- Damp.
- Zero Torque.
- High Stand / Low Stand.
- Set stand height.
- Set swing height.
- Velocity command.
- Move command.
- Continuous gait on / off.
- Next foot left / right.
- Wave hand.
- Shake hand / shake start / shake end.

12. Right-Wrist Control Surface

The right-wrist page focuses on RightWristYaw, motor index 26.

Capabilities:

- Live current q and dq readout.
- Absolute target command.
- Relative step command.
- Back-and-forth oscillation mode.
- Adjustable k_p , k_d , duration, period, and rate.
- Optional automatic gain selection.
- `arm_sdk` and `lowcmd` control path experimentation.
- Explicit arming and risk acknowledgement before command execution.
- Stop Wrist action.

This area is experimental and intentionally guarded.

The project includes code that can move a physical robot, so safety is treated as a product feature.

Current safeguards:

- Command validation on the server.
- Numeric limit checks for locomotion and wrist payloads.
- Fail-closed behavior when robot clients are unavailable.
- Explicit `armed=true` and `i_understand_risk=true` checks for wrist commands.
- Separate high-level loco path from high-risk low-level body control.
- Robot replay endpoint intentionally refuses physical playback until a safety controller exists.
- Production gate separates offline checks from live robot checks.
- README documents trusted-network requirements and unauthenticated HTTP risk.

The Recorder page supports two capture modes:

- Sequence recording.
- Pose point capture.

Sequence recording writes JSONL records under recordings/:

- recording_start
- telemetry_sample
- command_event
- recording_stop

Each sample can include:

- Wall-clock timestamp.
- Monotonic timestamp.
- Full body motor rows.
- IMU.
- Robot mode fields.
- Battery.
- Foot forces.

The Recorder page includes a replay viewer that is independent from the live robot command path.

Replay features:

- Recording file selector.
- Load, Play, Pause, scrub slider, and speed selector.
- JSONL sequence replay.
- Pose point replay with interpolation from live pose to target pose.
- Two-robot comparison:
- Translucent blue reference model from live robot state.
- Red recorded model from saved telemetry.
- Robot Play button unlocks only after the visual preview reaches the end.
- Server still returns a locked response for physical playback until a safety controller is implemented.

This gives operators a visual difference between the robot's current pose and a recorded or target motion.

The dashboard includes a CSV View for the current telemetry snapshot.

The repository also includes `tools/recording_to_csv.py`, which converts recordings into analysis-ready CSV tables:

- `samples.csv`
- `body_motors.csv`
- `hand_joints.csv`
- `imu.csv`
- `forces.csv`
- `events.csv`

The repository also includes `tools/generate_synthetic_recordings.py`, which creates replay-compatible synthetic data for testing without a live robot.

The dashboard can expose ROS graph information through /api/ros-graph.

The repository documents current robot control paths, including:

- /lowstate / rt/lowstate.
- /lowcmd / rt/lowcmd.
- /arm_sdk.
- /loco_sdk.
- /api/loco/request and /api/loco/response.
- /api/motion_switcher/request and response.
- Battery, mainboard, odometry, wireless controller, Inspire hand, video, and estimator topics.

This documentation is important because it prevents unsafe guessing when moving from observation to control.

The repository contains robot-side XR deployment automation:

- XR teleoperation checkout update.
- Vuer / XR launcher patching.
- Camera configuration patching.
- Image server patching.
- Dexterous retargeting patching.
- Inspire hand direct-curl patching.
- Left/right hand input swap patching.
- Head-tilt locomotion patching.
- Root children visual patching.
- Vuer locomotion pointer-events patching.
- Python requirement checks.

The deployment script installs and restarts all user-level services required for dashboard, camera, hands, XR, and watchdog operation.

Robot-side services:

- Dashboard server on port 8088.
- Telelmager camera server on port 60001.
- XR / Vuer service on port 8012.
- Inspire hand bridge.
- XR home watchdog.
- Auto-update timer from origin/main.

Operational helpers:

- `run_servers.py` starts the local dashboard.
- `kill_servers.py` stops local or robot-side dashboard processes.
- `deployment/install_robot_services.sh` installs the robot service set.
- `deployment/robot_autoupdate.sh` updates the robot checkout safely.

Logs are written under `/home/unitree/logs`.

The repository includes simulation and execution assets from the semantic teleoperation workspace:

- Unitree MuJoCo assets and scripts.
- Unitree ROS 2 support.
- Gazebo packages.
- MoveIt configuration.
- H1 visualization packages.
- H1 motion demo scripts.
- Real-robot guarded handshake script.

Vision Pro control also includes references to Unitree XR teleoperation and Isaac Lab simulation workflows.

This creates a path from offline replay and synthetic data into simulation validation before physical robot playback.

Quality and operations tooling includes:

- Unit / contract tests in tests/test_contracts.py.
- make test.
- make production-gate.
- Python syntax checks for owned scripts.
- Shell syntax checks for deployment scripts.
- JavaScript syntax checks for dashboard files.
- Contract checks for body joint ordering, hand joint ordering, wrist safety flags, command limits, loco actions, and recorder record ordering.

The production gate is intentionally offline by default, so it can run without robot access and without publishing robot commands.

Important limitations are explicit:

- The dashboard does not implement authentication.
- It must run only on trusted robot networks.
- Local development without Unitree SDK / CycloneDDS can test UI but not live telemetry.
- Raw trajectory playback to the physical robot is intentionally locked.
- Physical replay needs interpolation, joint limits, velocity limits, torque limits, controller ownership checks, emergency stop supervision, and simulation validation.
- Some arm and motion-switcher ownership details still require live robot investigation.
- No explicit project license is currently included.

The platform gives the team:

- A visible operator experience for demos.
- A live debugging console for robot telemetry.
- A safer path to supervised command testing.
- A reusable data capture pipeline.
- Replay and comparison tools for movement analysis.
- CSV export for analytics and ML preparation.
- XR teleoperation deployment automation.
- A consolidated workspace instead of scattered scripts.

This reduces setup friction and turns one-off robot experiments into repeatable workflows.

Near-term:

- Add authentication or network access control.
- Add clearer operator safety state and E-stop integration.
- Add richer recording metadata and session labels.
- Add CSV download directly from the dashboard.
- Add more tests around telemetry normalization and recorder conversion.

Robot playback readiness:

- Build a dedicated replay safety controller.
- Validate in simulation first.
- Add interpolation and limits.
- Add controller ownership checks.
- Add emergency stop supervision.
- Add supervised small-range physical tests.

Potential feature extensions:

- Recording library with tags, notes, operator name, and robot configuration.
- Motion comparison metrics between live and replay pose.
- Dataset export for imitation learning.
- Automatic anomaly detection for joint temperature, torque, voltage, and sample-rate drops.
- Web-based CSV download and charting.
- Pose-to-pose planner with speed and smoothness controls.
- Multi-camera view support.
- Cloud sync for selected non-sensitive recordings.
- Role-based dashboard access.
- Live health checklist before enabling command pages.

The project has moved from a telemetry viewer into a broader robot operator platform.

It now covers live state monitoring, XR operation, camera access, guarded control, full-body recording, digital twin replay, CSV data export, deployment automation, simulation assets, and offline validation.

The next major step is not adding raw power. It is adding production-grade safety, access control, and validated motion playback.